PathoFMPred: building and using portable pathology models

PathoFMPred authors

Scope and distribution

PathoFMPred stores many cancer-specific PLS regression and PLS-LDA fits in one portable object for each pathology slide representation. The object contains fitted parameters, feature order, development metrics, and provenance. It does not contain the original feature or outcome rows.

The CRAN-oriented package contains no fitted model, atlas registry, patient rows, or slide embeddings. A user can fit a new collection from authorized input tables or load a collection obtained separately under its applicable terms. No model is downloaded during installation, package loading, checking, or vignette construction.

The public package distributes no TITAN-, Giga-SSL- or Prov-GigaPath-derived fits. Users may build objects locally from authorized inputs; redistribution requires separate permission.

The PathoFMPred source code is MIT licensed. Separately obtained fitted objects and third-party inputs are not covered by that grant. The package licensing boundary is recorded in `inst/licenses/MODEL_ACCESS.md`.

All supplied performance estimates are internal research estimates. They do not establish external validity, assay equivalence, or clinical utility.

PathoFMPred checks that the installed fastPLS version matches the version recorded in each fitted object before inference. The model metadata also records software provenance, but installation from CRAN or another source does not change the prediction rule when the package version is the same.

Use a collection from an authorized source

```
library(PathoFMPred)
fetch_pathofmpred_models(
  foundation_model = "GigaSSL",
  base_url = "https://authorized.example.org/release",
  expected_sha256 = c(GigaSSL = "<SHA-256 supplied by the source>")
)
available_foundation_models()
```

The URL and digest are placeholders. The helper has no built-in model source. It requires a user-supplied source and verifies the supplied checksum before accepting an object. A user can instead place an authorized `pathofmpred_gigassl.rds` or `pathofmpred_provgigapath.rds` in the directory returned by `tools::R_user_dir("PathoFMPred", which = "data")`.

Build a PathoFMPred object from a new representation

The first three arguments are mandatory. Both tables must contain the column named by `id_column`. The outcome table must contain unique IDs. The feature table may contain repeated IDs, such as several slides from one patient. Repeated rows are pooled by patient mean by default or by patient median when `aggregation = "median"` is selected. More than 100 unique IDs must match.

```
features <- read.csv("foundation_model_features.csv", check.names = FALSE)
outcomes <- read.csv("patient_outcomes.csv", check.names = FALSE)

custom_object <- create_pathofmpred_object(
  feature_table = features,
  outcome_table = outcomes,
  id_column = "patient_id",
  foundation_model = "my_future_representation",
  aggregation = "mean",
  output_file = "pathofmpred_my_future_representation.rds"
)
```

The fitted collection can then be applied directly without adding it to an atlas registry. Repeated rows in a new feature table are pooled with the aggregation rule stored in the object.

```
new_features <- read.csv("new_foundation_model_features.csv",
                         check.names = FALSE)

custom_prediction <- predict_pathofmpred_object(
  object = custom_object,
  features = new_features,
  id_column = "patient_id"
)
custom_prediction[, c("patient_id", "endpoint", "outcome_type",
                      "prediction", "predicted_class_label", "lda_score")]
```

For a binary endpoint, `predicted_class_label` uses the negative and positive labels recorded during fitting. `lda_score` is an uncalibrated discriminant score and is not a probability.

The function reports feature-only IDs, outcome-only IDs, input-table row counts, and endpoint-specific missingness both before and after ID matching in `custom_object$data_audit`. Numeric outcomes with two observed levels are treated as binary by default. Other numeric outcomes are treated as continuous. Types can be fixed explicitly with a named vector. Binary outcomes encoded as anything other than `0/1` or `FALSE/TRUE` require an explicit event label through `positive_class`.

```
settings <- pathofmpred_control(
  components = 1:20,
  outer_folds = 5,
  inner_folds = 5,
  repeats = 5,
  method = "plssvd",
  binary_selection = "AUROC",
  seed = 2026
)

custom_object <- create_pathofmpred_object(
  features, outcomes, "patient_id",
  outcome_types = c(APC = "binary", Leukocyte_Fraction = "continuous"),
  positive_class = c(APC = "mutated"),
  control = settings
)
```

`rsvd_oversample` and `rsvd_power` are `NULL` by default. Therefore, PathoFMPred uses the installed fastPLS defaults. They can be overridden through `pathofmpred_control()` when a documented analysis requires different values.

Build TITAN, Giga-SSL and Prov-GigaPath objects locally

The following three recipes use the same package function. They require a patient-level outcome table with a unique `patient_id` column and at least 101 IDs in common with the selected representation. The table shown below as `patient_outcomes.csv` is an **input that users must create** from properly licensed source data, not a file bundled with PathoFMPred. It should contain one cancer type only, an ID column and selected endpoint columns. Do not pass the cancer label or other metadata as an outcome. Repeat the fitting call for each cancer; the generic builder does not automatically stratify cancers. For the project endpoint definitions and source-acquisition map, see the companion analysis repository. Its `R/run_all.R` pipeline is needed to reproduce the published atlas rather than fit an analogous custom collection.

The conversion commands shown here do not run during package installation or vignette construction. Run them only after acquiring the source files.

TITAN inputs and local fit

The official TITAN TCGA slide representations are provided by the TITAN authors in the gated Mahmood-Lab/TITAN repository, including `TCGA_TITAN_features.pkl`. Access must be requested from the upstream repository. The project conversion script is `tools/convert_tcga_titan_pickle.py` in the companion analysis repository. It creates a CSV whose rows retain TCGA slide identifiers and whose 768 feature columns are named consistently.

Outcome tables must be assembled independently. The companion analysis repository documents and converts the following sources:

- MC3 mutation calls from Ellrott et al. and cancer-driver annotations from Bailey et al.;
- PanImmune variables from Thorsson et al., including RNA-deconvolved cell fractions, transcriptomic signatures, methylation-derived leukocyte content, immune-repertoire quantities, in-silico neoantigens, and genomic-context variables;
- oncogenic pathway alterations from Sanchez-Vega et al.;
- aneuploidy and genome-doubling variables from Taylor et al.;
- gene fusions from Gao et al.;
- MSI fields from the cBioPortal TCGA PanCancer Atlas studies; and
- RNA pathway scores computed from TCGA expression matrices, including cancer hallmarks, glutathione, and vitamin B6 pathways.

The exact expected filenames and local configuration are recorded in `config/paths.example.R`. The acquisition and conversion workflow is documented in the companion `README`. Molecular sources do not all link to histology at the same level. Some provide a TCGA sample barcode, whereas the PanImmune table links at participant level. A matching sample barcode does not prove that the slide and molecular assay came from the same block, portion, analyte, or tumour region.

After conversion, a local TITAN collection can be fitted as follows. This code is intentionally not executed while the vignette is built.

```
titan_features <- read.csv(
  "TCGA_TITAN_features.csv", check.names = FALSE
)
endpoint_table <- read.csv("patient_outcomes.csv", check.names = FALSE)

pathofmpred_titan <- create_pathofmpred_object(
  feature_table = titan_features,
  outcome_table = endpoint_table,
  id_column = "patient_id",
  foundation_model = "TITAN",
  aggregation = "mean",
  control = pathofmpred_control(
```

```
    components = 1:20,
    outer_folds = 5,
    inner_folds = 5,
    repeats = 5,
    method = "plssvd",
    binary_selection = "AUROC"
  ),
  output_file = "pathofmpred_titan_coad.rds"
)

titan_prediction <- predict_pathofmpred_object(
  object = pathofmpred_titan,
  features = read.csv("new_TITAN_features.csv", check.names = FALSE),
  id_column = "patient_id"
)
```

Keep `pathofmpred_titan_coad.rds` local or in the access-controlled `tkcaccia/PathoFMPred-private` repository. It must not be added to the public `tkcaccia/PathoFMPred` repository under the current release policy.

Giga-SSL inputs and local fit

The Giga-SSL repository links to the published TCGA slide embeddings in `TCGA_encoded_t5e1000`. Download the upstream `ids.npy` and `embeddings.npy` files from the source linked there. The companion analysis repository's `tools/convert_foundation_model_embeddings.py` converts them to 512 numeric feature columns and a slide `filename` column:

```
python tools/convert_foundation_model_embeddings.py \
  --model gigassl \
  --source /path/to/TCGA_encoded_t5e1000 \
  --output GigaSSL_TCGA_features.csv.gz \
  --manifest GigaSSL_conversion.json
```

```
gigassl_features <- read.csv("GigaSSL_TCGA_features.csv.gz",
                             check.names = FALSE)
gigassl_features$patient_id <- substr(gigassl_features$filename, 1L, 12L)
gigassl_features$filename <- NULL

gigassl_object <- create_pathofmpred_object(
  feature_table = gigassl_features,
  outcome_table = endpoint_table,
  id_column = "patient_id",
  foundation_model = "GigaSSL",
  aggregation = "mean",
  control = pathofmpred_control(
    components = 1:20, outer_folds = 5, inner_folds = 5,
    repeats = 5, method = "plssvd", binary_selection = "AUROC"
  ),
  output_file = "pathofmpred_gigassl_coad.rds"
)
```

Prov-GigaPath inputs and local fit

Obtain the Parquet file from the TCGA Prov-GigaPath embedding dataset under its stated terms. The same conversion script selects slide-encoder layer index 13, verifies exact duplicate slide vectors and retains one row per slide. It produces 768 numeric feature columns.

```
python tools/convert_foundation_model_embeddings.py \
  --model provgigapath \
  --source /path/to/tcga_provgigapath_embeddings.parquet \
  --output ProvGigaPath_TCGA_final_layer_features.csv.gz \
  --manifest ProvGigaPath_conversion.json \
  --provgigapath-layer 13
```

```
prov_features <- read.csv("ProvGigaPath_TCGA_final_layer_features.csv.gz",
                          check.names = FALSE)
prov_features$patient_id <- substr(prov_features$filename, 1L, 12L)
prov_features$filename <- NULL

prov_object <- create_pathofmpred_object(
  feature_table = prov_features,
  outcome_table = endpoint_table,
  id_column = "patient_id",
  foundation_model = "ProvGigaPath",
  aggregation = "mean",
  control = pathofmpred_control(
    components = 1:20, outer_folds = 5, inner_folds = 5,
    repeats = 5, method = "plssvd", binary_selection = "AUROC"
  ),
  output_file = "pathofmpred_provgigapath_coad.rds"
)
```

Before fitting, verify that every extracted ID is a valid TCGA participant barcode and that the outcome table uses the same ID level. Repeated slide rows are pooled within patient, so slides from one participant do not create extra training observations. The returned `data_audit` records unmatched IDs and outcome-specific missingness. Keep the three fitted objects local unless their redistribution rights are established in writing.

Use an authorized atlas collection

The following example works after a Giga-SSL research collection has been installed separately under its applicable terms. It is not executable from a fresh CRAN installation.

```
models <- available_pathofm_models("GigaSSL", cancer = "COAD")
models[, c("endpoint", "outcome_type", "n")]

x <- example_pathofm_features("COAD", foundation_model = "GigaSSL")
prediction <- predict_pathofm(
  cancer = "COAD",
  features = x,
  foundation_model = "GigaSSL",
  patient_id = "software_example"
)
prediction[, c("endpoint", "prediction", "predicted_class", "lda_score")]
```

Binary scores and reference ranks are not probabilities or clinical-risk estimates. Metadata distinguish primary matched-cohort performance from five-repeat estimates; the latter are absent when repeats were not run.